



UNIVERSIDAD
NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE CIENCIAS

Preguntas práctica 4

Integrantes:

Yonathan Berith Jaramillo Ramírez. 419004640

Profesor: Adrián Ulises Mercado Martínez

Ayudantes: Yessica Janeth Pablo Martínez

Carlos Gerardo Acosta Hernández

2 Dec, 2024

Respuestas a las Preguntas Teóricas

4. Conjuntos N , Σ y símbolo inicial S

Dado el análisis de la gramática proporcionada, los conjuntos se definen como:

$$N = \{S, Expr, Expr', Term, Term', Factor, Num, Decimal, Entero, Digito, Asig, Var, Pos, Letra\}$$

$$\Sigma = \{+, -, *, /, (,), =, ., 0, 1, \dots, 9, \text{var, if, else, while}\}$$

$$S = \{S\}$$

5. Eliminación de ambigüedad

La gramática es ambigua debido a que no establece precedencia o asociatividad para operadores. Por ejemplo, $Expr \rightarrow Expr + Expr$ permite derivaciones múltiples para expresiones con más de un operador.

Para resolver esto:

- Se define precedencia explícita para los operadores.
- Se asegura que las operaciones más fuertes (multiplicación y división) tengan mayor precedencia que las más débiles (suma y resta).
- Se respeta la asociatividad a la izquierda de los operadores aritméticos.

Gramática ajustada:

$$\begin{aligned} Expr &\rightarrow Expr + Term \mid Expr - Term \mid Term \\ Term &\rightarrow Term * Factor \mid Term / Factor \mid Factor \\ Factor &\rightarrow (Expr) \mid Var \mid Num \end{aligned}$$

6. Eliminación de recursividad izquierda

Identificamos recursividad izquierda en:

$$\begin{aligned} Expr &\rightarrow Expr + Term \mid Expr - Term \mid Term \\ Term &\rightarrow Term * Factor \mid Term / Factor \mid Factor \end{aligned}$$

Eliminamos la recursividad izquierda reescribiendo:

$$\begin{aligned} Expr &\rightarrow TermExpr' \\ Expr' &\rightarrow +TermExpr' \mid -TermExpr' \mid \varepsilon \\ Term &\rightarrow FactorTerm' \\ Term' &\rightarrow *FactorTerm' \mid /FactorTerm' \mid \varepsilon \end{aligned}$$

7. Factorización izquierda

La gramática no presenta prefijos comunes en las producciones, por lo que no es necesario aplicar factorización izquierda.

8. Nuevos conjuntos N y P

Tras los ajustes realizados, los conjuntos actualizados son:

$$N = \{S, Expr, Expr', Term, Term', Factor, Num, Asig, Var, Pos, Letra\}$$

Producciones actualizadas:

$$\begin{aligned} S &\rightarrow Expr \mid Asig \\ Expr &\rightarrow TermExpr' \\ Expr' &\rightarrow +TermExpr' \mid -TermExpr' \mid \varepsilon \\ Term &\rightarrow FactorTerm' \\ Term' &\rightarrow *FactorTerm' \mid /FactorTerm' \mid \varepsilon \\ Factor &\rightarrow (Expr) \mid Var \mid Num \\ Asig &\rightarrow \text{var}Var = Expr \\ Var &\rightarrow LetraPos \\ Pos &\rightarrow Var \mid \varepsilon \\ Letra &\rightarrow _ \mid a \mid \dots \mid z \mid A \mid \dots \mid Z \end{aligned}$$

9. Manejo de conflictos shift/reduce

El problema de conflictos shift/reduce se presenta cuando el analizador sintáctico no puede decidir si debe desplazar un símbolo de entrada o reducir una producción. Esto ocurre frecuentemente en gramáticas con operadores que tienen múltiples niveles de precedencia o en estructuras como ‘if-else’ debido al problema del “dangling else”.

Para resolver estos conflictos, se realizaron los siguientes ajustes:

1. **Precedencia y asociatividad de operadores:** Se añadieron declaraciones explícitas para los operadores aritméticos y de comparación, definiendo su precedencia y asociatividad mediante las directivas de Yacc:

```
%left '+' '-'
%left '*' '/'
%nonassoc '(' ')'
%nonassoc '<' '>' '<=' '>=' '==' '!='
```

Esto asegura que el analizador respete el orden de las operaciones matemáticas, dando mayor prioridad a ‘*’ y ‘/’ sobre ‘+’ y ‘-’. Además, los operadores de comparación no son asociativos, lo que evita ambigüedades en expresiones como ‘a ; b == c’.

2. **Manejo del *dangling else*:** Se resolvió este conflicto mediante la asignación de una precedencia especial al token ‘THEN’. Esto garantiza que el ‘else’ se asocie correctamente al ‘if’ más cercano:

```
%prec THEN
```

La regla para ‘if-else’ queda definida como:

```
if_stmt : IF '(' expresion ')' sentencias %prec THEN
        | IF '(' expresion ')' sentencias ELSE sentencias ;
```

3. **Reestructuración de la gramática:** Se eliminaron reglas con epsilon y se incluyeron producciones explícitas para bloques “ y listas de sentencias. Esto reduce la necesidad de decisiones ambiguas durante el análisis.

Gramática Final

A continuación, se presenta la gramática resultante tras todos los ajustes realizados:

```

programa → lista_declaraciones cuerpo_programa EOF
cuerpo_programa → bloque_principal | lista_sentencias
bloque_principal → {lista_sentencias}
lista_declaraciones → lista_declaraciones declaracion | declaracion
declaracion → tipo lista_var ;
tipo → int | float
lista_var → lista_var , id | id
lista_sentencias → lista_sentencias sentencia | sentencia
sentencias → {lista_sentencias} | sentencia
sentencia → asignacion | if_stmt | while_stmt
asignacion → id = expresion ;
if_stmt → if (expresion) sentencias | if (expresion) sentencias else sentencias
while_stmt → while (expresion) sentencias
expresion → expresion + termino | expresion - termino | termino
termino → termino * factor | termino / factor | factor
factor → (expresion) | id | numero_entero | numero_real

```

Esta gramática resuelve todos los conflictos shift/reduce mediante una estructura clara y reglas explícitas que priorizan la determinación de precedencia y asociatividad. También incluye producciones que permiten el manejo de bloques y sentencias condicionales o iterativas.
